

Data Structures and Algorithms

Instructor: Dr. Hao Qin (秦浩)

E-mail: hao.qin@scupi.cn

My Office: Room 514, SCUPI (North)

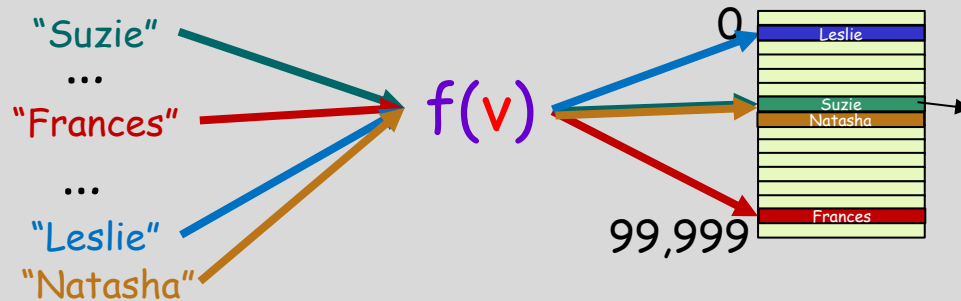
Hash Tables

What's the big picture?

Hash tables offer **faster searching** than a BST, but they don't order their elements alphabetically.

Create an array with N slots to hold your values.

Define a func `f()` that takes a value `V` as an input and produces a slot # from 0 to N-1 as its output.



Use $f()$ to find the right slot # to either add a new item, or search for an existing item.

f() will map several items to the same slot. You can either place the 2nd item in the next open slot or have a linked list of items in each slot.



e.g., for people, $f()$ might add the person's birthday digits mm+dd+yyyy to compute a slot #.

The Modulus Operator

In C++, the **% operator** is used to divide two numbers and obtain the **remainder**.

For example, if we compute:

```
int x = 1234 % 100;
```

the value of x will be **34**.

$$\begin{array}{r} 12 \text{ R} \\ 100 \overline{) 1234} \\ \underline{100} \\ 234 \\ \underline{200} \\ 34 \end{array}$$

Rule: When you divide by a given value **N**, all of your **remainders** are guaranteed to be **between 0 and N-1!**

The (Almost) Hash Table

How can we create an ADT where we can insert the 9-digit student ID#s for all 50,000 students...

and then find if our ADT holds a given ID#
in just **one algorithmic step**?!?!?

That can't be done... can it?

It can, and let's see how!

Let's use a **really, really large array** to hold our #s.

The (Almost) Hash Table

```

class AlmostHashTable
{
public:
    void addItem(int n)
    {
        → m_array[n] = true;
    }
    → bool holdsItem(int q)
    {
        → return m_array[q] == true;
    }
private:
    bool m_array[100000000]; // big!
};

int main()
{
    → AlmostHashTable x;

    → x.addItem(400683948);
    → if (x.holdsItem(1234) != true)
        → cout<< "Couldn't find it!";
}

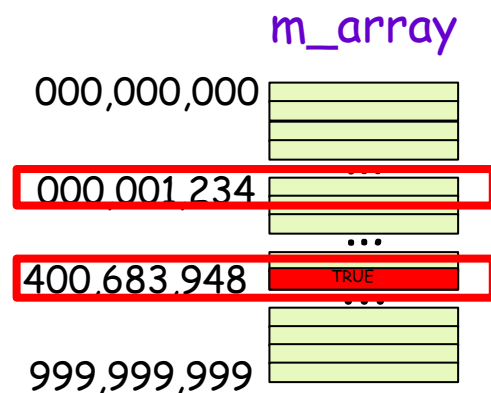
```

Idea:

Let's create an array with **1 billion slots** - one slot for each valid ID#.

To **add a new ID#** with a value of **N**, we'll simply set **array[N]** to true.

To determine if our array **holds a previously-added value Q**, simply check if **array[Q]** is true.



The (Almost) Hash Table

OK - so now we know how to build an $O(1)$ search!
But what's the problem with our ADT?

It's really, really inefficient:
Our array has 1 billion slots
yet there are only 50,000 student IDs
we could possibly add to it,
so we're wasting 999,950,000 of the slots...

It would be great if we could use the same algorithm
but with a smaller array, say one with 100,000 slots
instead of 1 billion!

The Mapping Function

How can we write a `mapFunc` that converts our large `ID#` into a `bucket #` that falls within our 100,000 element array?

```
int mapFunc(int idNum)
{
    const int ARRAY_SIZE = 100000;
    int bucket = idNum % ARRAY_SIZE;
    return bucket;
}
```

This line takes an input value `idNum` and returns an output value between `0` and `ARRAY_SIZE - 1`.
(0 to 99,999)

And this corresponding value can be used to pick a bucket in our 100,000 element array!

RIGHT! The C++ `%` operator (aka the **modulus division operator**) does exactly what we want!!!

So now for each input `ID#` we can compute a corresponding value between **0-99,999!**

The (Almost) Hash Table

Let's see how it works.

```
class AlmostHashTable2
{
public:
→ void addItem(int n)
{
→ int bucket = mapFunc(n);
→ m_array[bucket] = true;
→ }

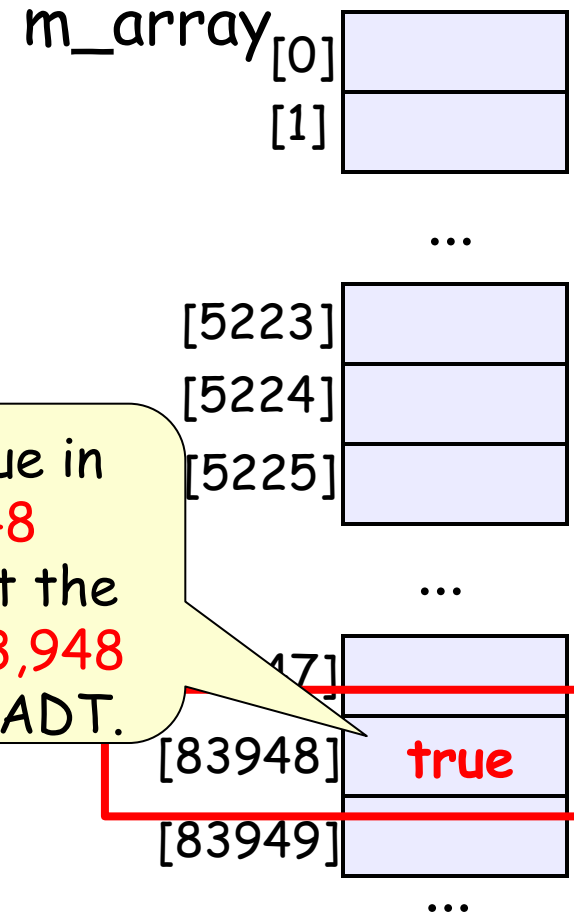
private:
int mapFunc(int idNum)
{
return idNum % 100000;
}

bool m_array[100000]; // not
};
```

$$400,683,948 \% 100,000 = 83,948$$

```
int main()
{
→ AlmostHashTable2 x;
→ x.addItem(400683948);
x.addItem(111105224);
x.addItem(222205224);
}
```

The **true** value in slot **83,948** indicates that the value **400,683,948** is held in our ADT.



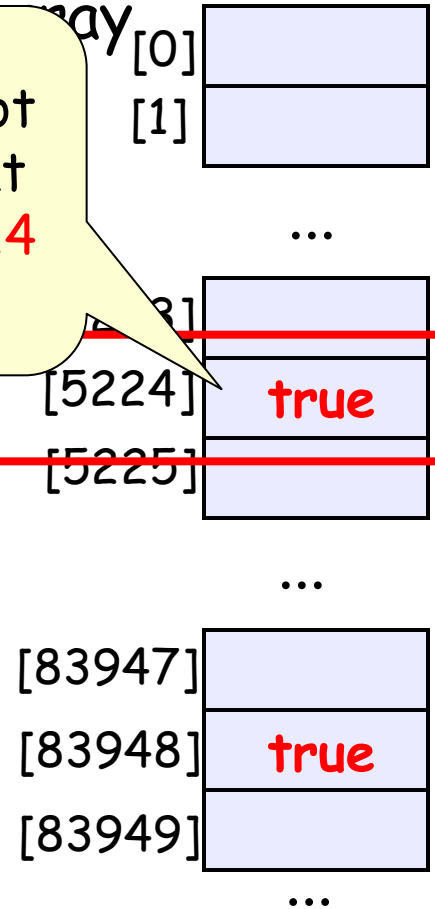
The (Almost) Hash Table

Let's see how it works.

```
class AlmostHashTable2
{
public:
→ void addItem(int
    {
→ int bucket = mapFunc(n);
→ m_array[bucket] = true;
→ }
private:
    int mapFunc(int idNum)
    {
        return idNum % 100000;
    }
    bool m_array[100000]; // not so big!
};
```

$111,105,224 \% 100,000 =$
5,224

The **true** value in slot **5,224** indicates that the value **111,105,224** is held in our ADT.



```
int main()
{
    AlmostHashTable2 x;
    x.addItem(400683948);
→ x.addItem(111105224);
    x.addItem(222205224);
}
```

(Almost) Hash Table

But our mapping function wants to also put a **true** value in slot **5,224** to represent **222,205,224**!

$$222,205,224 \% 100,000 = 5,224$$

Ok, let's add the last ID# to our table...

```

→ m_array[5224] = true;
}

private:
int mapFunc(int idNum)
{
    return idNum % 100000;
}

bool m_array[100000]; // not
};

```

But wait! We already stored a **true** value in bucket **5,224** to represent value **111,105,224**.

This is called a **collision**!

[5224] **true**

[5225]

```

int main()
{
    AlmostHashTable2 x;
    x.addItem(400683948);
    x.addItem(111105224);
    → x.addItem(222205224);
}

```

How can I tell if my hash table holds **222,205,224** or **111,105,224**?

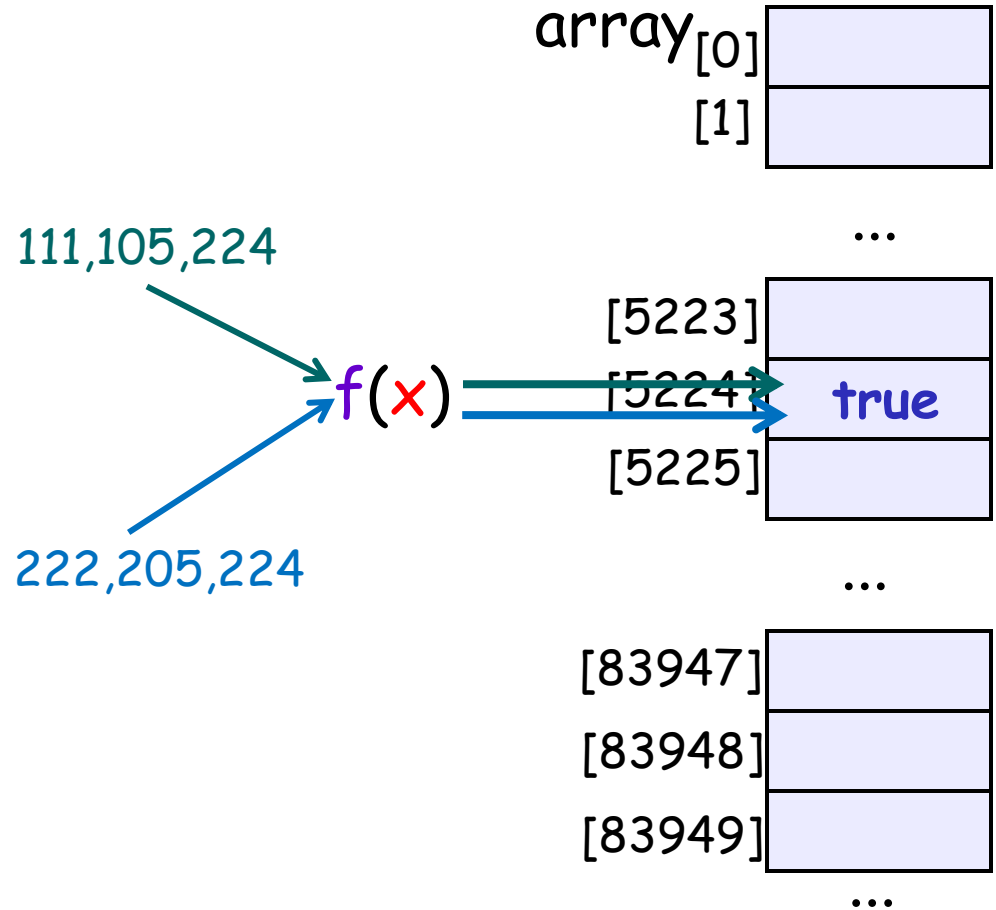
true

The (Almost) Hash Table: **A problem!**

A **collision** is a condition where **two or more values** both map to the same bucket in the array.

we can't tell what value was actually stored in the array!

Let's see how to fix this problem!



Closed Hash Table with Linear Probing: Insertion

Linear Probing Insertion:

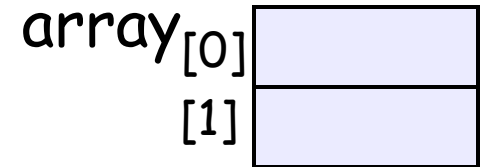
As before, we use our mapping function to locate the right bucket in our array.

If the target bucket is empty, we can store our value there.

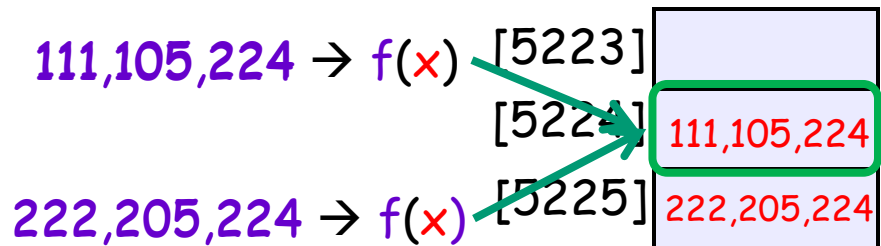
However, instead of storing **true** in the bucket, we store our **full original value** - this prevents ambiguity!

If the bucket is occupied, scan down from that bucket until we hit the first open bucket. Put the new value there.

This bucket was already **filled**, so we can't put our value here! Let's scan down for an open spot.



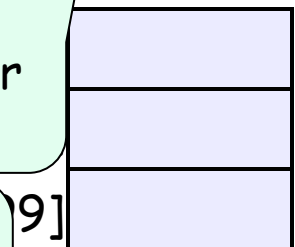
...



This next bucket is **empty**, so we can put our new value here!

This bucket is currently **empty**, so we can put our new value here.

...



Closed Hash Table with Linear Probing: Searching

Linear Probing Searching:

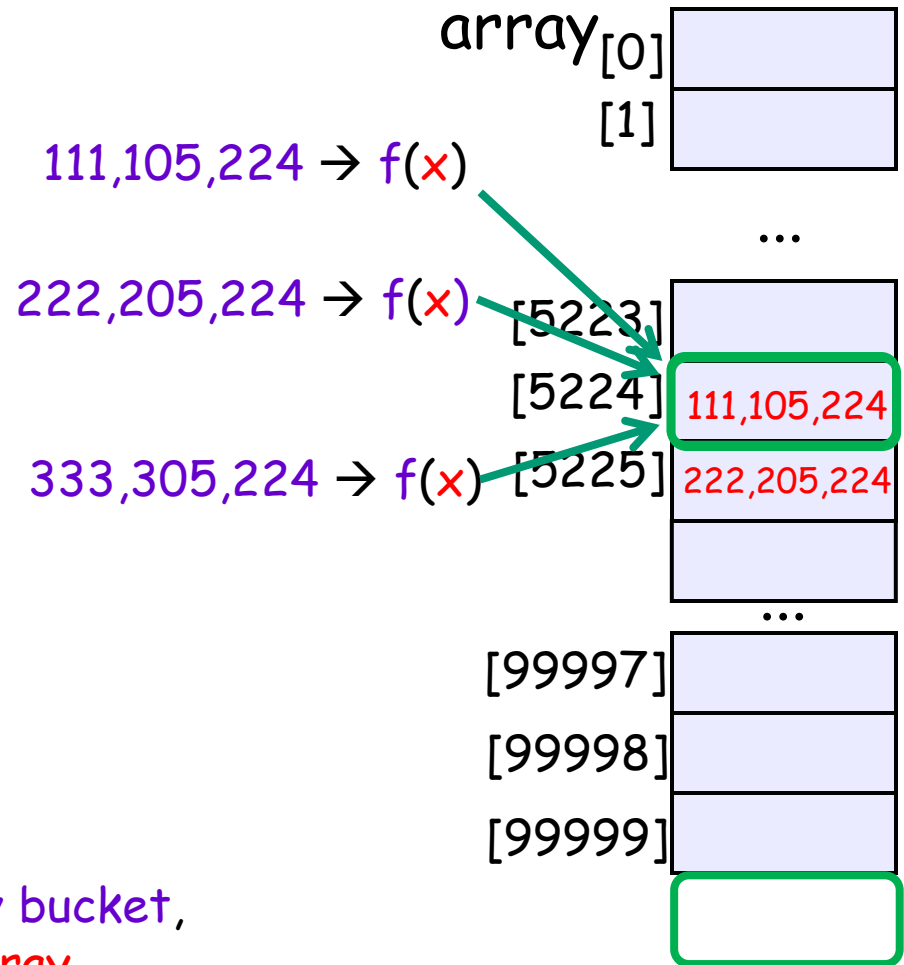
To search our hash table, we use a similar approach.

We compute a target bucket number with our mapping function.

We then look in that bucket for our value. If we find it, great!

If we don't find our value, we scan linearly down the array until we either find our value or hit an empty bucket.

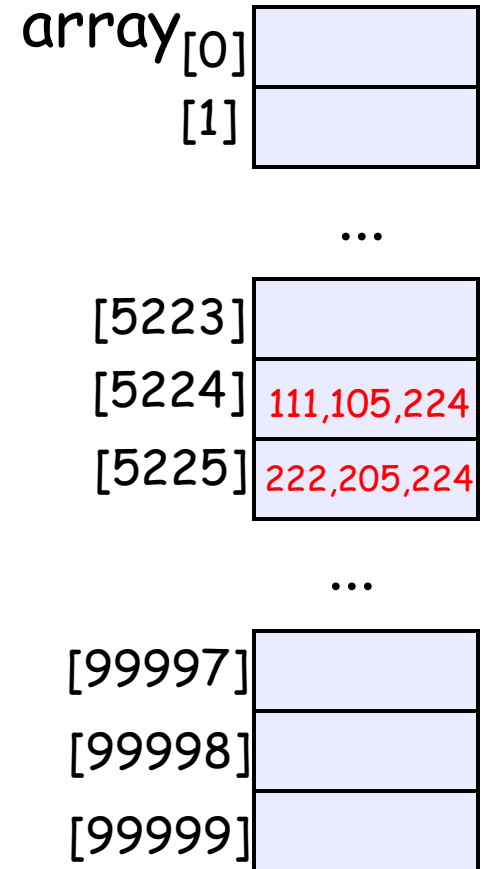
If while scanning, you run into an empty bucket, it means: your value isn't in the array.



Closed Hash Table with Linear Probing

This approach addresses collisions by putting each value as close as possible to its intended bucket.

Since we store every original value (e.g., 111,105,224) in the array, there is no chance of ambiguity.



Closed Hash Table with Linear Probing

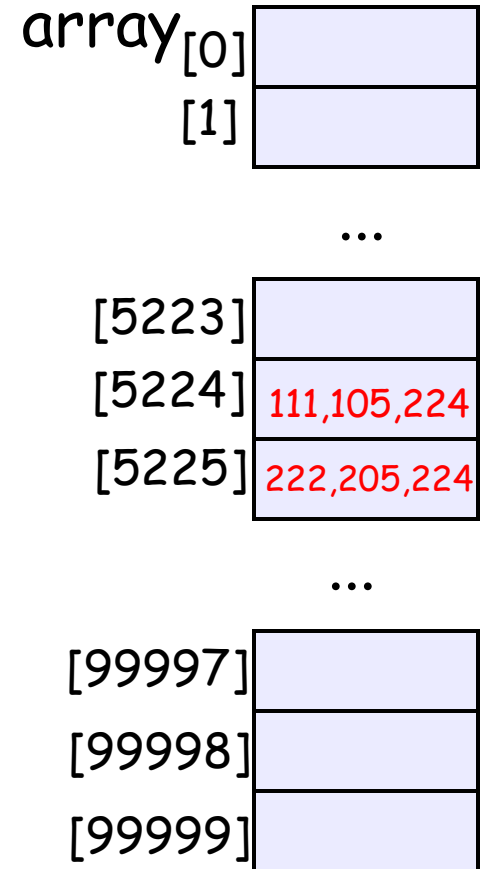
So why do we call this a
"Closed" hash table???

Since our data is stored in a
fixed-size array, there are a
fixed (closed) number of buckets
for us to put values.

Once we run out of empty buckets,
we can't add new values...

Linked lists and binary search trees
don't have this problem!

Ok, let's see the C++ code now!



16

```
const int NUM_BUCKET = 10;

class HashTable
{
public:
    void insert(int idNum)
    {
        int bucket = mapFunc(idNum);

        for (int tries=0; tries<NUM_BUCKET; tries++)
        {
            if (m_buckets[bucket].used == false)
            {
                m_buckets[bucket].idNum = idNum;
                m_buckets[bucket].used = true;
                return;
            }
            bucket = (bucket + 1) % NUM_BUCKET;
        }
        // no room left in hash table!!!
    }

private:
    int mapFunc(int idNum) const
    { return idNum % NUM_BUCKET; }

    BUCKET m_buckets[NUM_BUCKET];
};
```

Linear Probing: Inserting

17 const int NUM_BUCK = 10

class HashTable

{
public:

→ void insert(int idNum)

{

→ int bucket = mapFunc(idNum);

→ for (int tries=0; tries<NUM_BUCK; tries++)

{

→ if (m_buckets[bucket].used == false)

{

→ m_buckets[bucket].idNum = idNum;

→ m_buckets[bucket].used = true;

→ return;

}

bucket = (bucket + 1) % NUM_BUCK;

}

// no room left in hash table!!!

}

private:

int mapFunc(int idNum) const

{ return idNum % NUM_BUCK; }

BUCKET m_buckets[NUM_BUCK];

};

bucket = 29 % NUM_BUCK
bucket = 29 % 10
bucket = 9

29

bucket 9

near Probing:
Inserting

0	idNum:		used:	f
1	idNum:		used:	f
2	idNum:		used:	f
3	idNum:		used:	f
4	idNum:		used:	f
5	idNum:		used:	f
6	idNum:		used:	f
7	idNum:		used:	f
8	idNum:		used:	f
9	idNum:	29	used:	t

main()

{

→ HashTable ht;

→ ht.insert(29);

→ ht.insert(65);

ht.insert(79);

}

18 `const int NUM_BUCK = 10`

`class HashTable`

`{`
`public:`

→ `void insert(int idNum)`

`{`

→ `int bucket = mapFunc(idNum);`

→ `for (int tries=0; tries<NUM_BUCK; tries++)`

`{`

→ `if (m_buckets[bucket].used == false)`

`{`

→ `m_buckets[bucket].idNum = idNum;`

→ `m_buckets[bucket].used = true;`

→ `return;`

`}`

`bucket = (bucket + 1) % NUM_BUCK;`

`}`

`// no room left in hash table!!!`

`}`

`private:`

`int mapFunc(int idNum) const`

`{ return idNum % NUM_BUCK; }`

`BUCKET m_buckets[NUM_BUCK];`

`};`

65

bucket = 65 % NUM_BUCK
bucket = 65 % 10
bucket = 5

bucket 5

near Probing:
Inserting

0	idNum:		used:	f
1	idNum:		used:	f
2	idNum:		used:	f
3	idNum:		used:	f
4	idNum:		used:	f
5	idNum:	65	used:	T
6	idNum:		used:	f
7	idNum:		used:	f
8	idNum:		used:	f
9	idNum:	29	used:	T

`main()`

`{`

`HashTable ht;`

`ht.insert(29);`

→ `ht.insert(65);`

→ `ht.insert(79);`

`}`

19 `const int NUM_BUCK = 10;`

`class HashTable`

`{`
`public:`

→ `void insert(int idNum)`

`{`

→ `int bucket = mapFunc(idNum);`

→ `for (int tries=0; tries<NUM_BUCK; tries++)`

`{`

→ `if (m_buckets[bucket].used == false)`

`{`

→ `m_buckets[bucket].idNum = idNum;`

→ `m_buckets[bucket].used = true;`

→ `return;`

`}`

→ `bucket = (bucket + 1) % NUM_BUCK;`

`}`

`// no room left in hash table!!!`

`}`

`private:`

`int mapFunc(int idNum) const`

`{ return idNum % NUM_BUCK; }`

`BUCKET m_buckets[NUM_BUCK];`

`};`

`bucket = 79 % NUM_BUCK`
`bucket = 79 % 10`
`bucket = 9`

79

bucket

9

Linear Probing: Inserting

0	idNum: 79	used: T
1	idNum:	used: f
2	idNum:	used: f
3	idNum:	used: f
4	idNum:	used: f
5	idNum: 65	used: T
6	idNum:	used: f
7	idNum:	used: f
8	idNum:	used: f
9	idNum: 29	used: T

`main()`

`{`

`HashTable ht;`

`ht.insert(29);`

`ht.insert(65);`

→ `ht.insert(79);`

`}`

```
const int NUM_BUCK = 10;

class HashTable
{
public:
    bool search(int idNum)
    {
        int bucket = mapFunc(idNum);
        for (int tries=0;tries<NUM_BUCK;tries++)
        {
            if (m_buckets[bucket].used == false)
                return false;
            if (m_buckets[bucket].idNum == idNum)
                return true;

            bucket = (bucket + 1) % NUM_BUCK;
        }
        return false;// not in the hash table
    }

private:
    int mapFunc(int idNum) const
    {   return idNum % NUM_BUCK;   }

    BUCKET m_buckets[NUM_BUCK];
};
```

Linear Probing: Searching

```
const int NUM_BUCKET = 10;
```

```
class HashTable
```

```
{
```

```
public:
```

```
→ bool search(int idNum)
```

```
{
```

```
→ int bucket = mapFunc(idNum);
```

```
→ for (int tries=0; tries<NUM_BUCKET; tries++)
```

```
{
```

```
→ if (m_buckets[bucket].used == false)  
    return false;
```

```
→ if (m_buckets[bucket].idNum == idNum)  
    → return true;
```

```
    bucket = (bucket + 1) % NUM_BUCKET;
```

```
}
```

```
return false; // not in the hash table
```

```
}
```

```
private:
```

```
int mapFunc(int idNum) const
```

```
{ return idNum % NUM_BUCKET; }
```

```
BUCKET m_buckets[NUM_BUCKET];
```

```
};
```

bucket = 29 % NUM_BUCKET
bucket = 29 % 10
bucket = 9

29

bucket 9

idNum:	79	used:	T
idNum:		used:	f
idNum:		used:	f
idNum:		used:	f
idNum:		used:	f
idNum:	65	used:	T
idNum:	15	used:	T
idNum:	175	used:	T
idNum:		used:	f
idNum:	29	used:	T

```
main()
```

```
{
```

```
→ HashTable ht;
```

```
→ ...
```

```
→ bool x;
```

```
→ x = ht.search(29);
```

```
→ x = ht.search(175);
```

```
    x = ht.search(20);
```

```
}
```

```
const int NUM_BUCK = 10;
```

```
class HashTable
```

```
{
```

```
public:
```

```
→ bool search(int idNum)
```

```
{
```

```
→ int bucket = mapFunc(idNum);
```

```
→ for (int tries=0;tries<NUM_BUCK;tries++)
```

```
{
```

```
→ if (m_buckets[bucket].used == false)  
    return false;
```

```
→ if (m_buckets[bucket].idNum == idNum)  
    → return true;
```

```
→ bucket = (bucket + 1) % NUM_BUCK;
```

```
→ }
```

```
    return false;// not in the hash table
```

```
}
```

```
private:
```

```
    int mapFunc(int idNum) const
```

```
    { return idNum % NUM_BUCK; }
```

```
    BUCKET m_buckets[NUM_BUCK];
```

```
};
```

bucket = 175 % NUM_BUCK
bucket = 175 % 10
bucket = 5

175

bucket

5

	idNum:	79	used:	T
	idNum:		used:	f
	idNum:		used:	f
	idNum:		used:	f
4	idNum:		used:	f
5	idNum:	65	used:	T
6	idNum:	15	used:	T
7	idNum:	175	used:	T
8	idNum:		used:	f
9	idNum:	29	used:	T

```
main()
```

```
{
```

```
    HashTable ht;
```

```
    ...
```

```
    bool x;
```

```
    x = ht.search(29);
```

```
→ x = ht.search(175);
```

```
→ x = ht.search(20);
```

```
}
```

```
const int NUM_BUCKET = 10;
```

```
class HashTable
```

```
{
```

```
public:
```

```
→ bool search(int idNum)
```

```
{
```

```
→ int bucket = mapFunc(idNum);
```

```
→ for (int tries=0; tries<NUM_BUCKET; tries++)
```

```
{
```

```
→ if (m_buckets[bucket].used == false)
```

```
→ return false;
```

```
→ if (m_buckets[bucket].idNum == idNum)
```

```
    return true;
```

```
→ bucket = (bucket + 1) % NUM_BUCKET;
```

```
→ }
```

```
    return false; // not in the table
```

```
}
```

```
private:
```

```
    int mapFunc(int idNum) const
```

```
    { return idNum % NUM_BUCKET;
```

```
    BUCKET m_buckets[NUM_BUCKET];
```

```
};
```

bucket = 20 % NUM_BUCKET
bucket = 20 % 10
bucket = 0

bucket 0

idNum:	79	used:	T
idNum:		used:	T
idNum:		used:	f
idNum:		used:	f
idNum:		used:	f
idNum:		used:	f
idNum:	65	used:	T
idNum:	15	used:	T
idNum:	175	used:	T
idNum:		used:	f
idNum:	29	used:	T

The bucket is empty. This means that the value (20) we're searching for can't possibly be in the table. If it were in the table, we'd have already found it before hitting an empty slot!

```
x = ht.search(175);
```

```
→ x = ht.search(20);
```

```
}
```

The "Open Hash Table"

We just saw how to use **linear probing** to deal with collisions in our **closed hash table**.

Our **closed hash table** + **linear probing** works just fine, but it still has a few problems:

It's **difficult** to **delete items**

It has a **cap on the number of items** it can hold...

It'd be nice if we could find a way to avoid both of these problems, yet still have an $O(1)$ table!

We can! And it's called the "**Open Hash Table**."
Let's see how it works!

The "Open" Hash Table

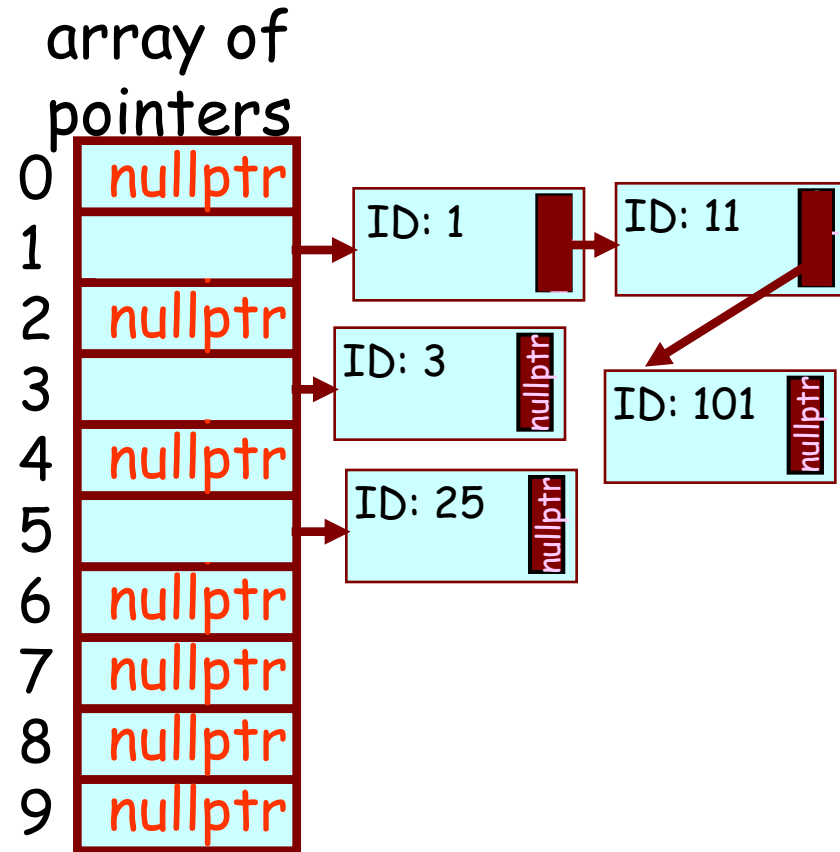
Idea: Instead of storing our values directly in the array, each array bucket points to a linked list of values.

To search for an item:

1. As before, compute a bucket # with your mapping function:

```
bucket = mapFunc(idNum);
```

2. Search the linked list at `array[bucket]` for your item
3. If we reach the end of the list without finding our item, it's not in the table!



Insert the following values: 1, 3, 11, 25, 101

So basically it's a tradeoff!

You could always use a **really big hash table** with **way-too-many buckets** and ensure **really fast searches**...

But then you'll end up **wasting lots of memory**...

On the other hand, if you have a **really small hash table** (with just barely enough room), it'll **be slower**.

Finally, when **choosing the exact size** of your hash table (the number of buckets)...

Choosing a Hash Function: Tips

1. The hash function must always give us the same output value for a given input value:

Today: $\text{hash}(400683948) \rightarrow 83,948$

Tomorrow: $\text{hash}(400683948) \rightarrow \textit{still } 83,948$

2. The hash function should disperse items throughout the hash array as randomly as possible.

$\text{hash}(\text{abc}) = 294$

$\text{hash}(\text{cba}) = 294$

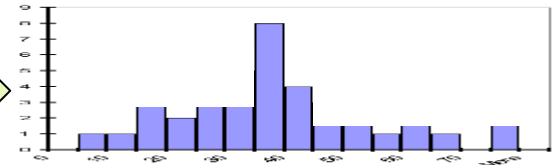
Not good!

3. When coming up with a new hash function, always measure how well it disperses items (do some experiments!)



Good!

Bad!



Practice1: Designing a Simple Hash Table

A hash table is a data structure that allows you to store and retrieve values based on a key. It uses a hash function to compute an index into an array of buckets or slots, from which the desired value can be found. Here's a step-by-step guide to designing a simple hash table in C++:

Components of a Hash Table

Array (or vector): This is the underlying data structure that holds the buckets.

Hash Function: This function takes a key and converts it into an index for the array.

Collision Resolution: A method to handle cases where two keys hash to the same index.

Solution to Practice1

```
#include <iostream>
#include <vector>
#include <list>
#include <utility> // For std::pair

class SimpleHashTable {
private:
    std::vector<std::list<std::pair<int, std::string>>> table;
    int table_size;

    int hash(int key) {
        return key % table_size;
    }

int main() {
    SimpleHashTable ht(10);
    ht.insert(1, "Alice");
    ht.insert(2, "Bob");
    std::cout << "Get 1: " << ht.get(1) << std::endl; // Output: Alice
    std::cout << "Get 2: " << ht.get(2) << std::endl; // Output: Bob
    ht.remove(1);
    std::cout << "Get 1 after removal: " << ht.get(1) << std::endl; // Output: Not found
    return 0;
}
```

```
public:
    SimpleHashTable(int size) : table_size(size) {
        table.resize(table_size);
    }

    void insert(int key, const std::string& value) {
        int index = hash(key);
        for (auto& pair : table[index]) {
            if (pair.first == key) {
                pair.second = value; // Update value if key exists
                return;
            }
        }
        table[index].push_back({key, value});
    }

    std::string get(int key) {
        int index = hash(key);
        for (const auto& pair : table[index]) {
            if (pair.first == key) {
                return pair.second;
            }
        }
        return "Not found"; // Key not found
    }

    void remove(int key) {
        int index = hash(key);
        for (auto it = table[index].begin(); it != table[index].end();
++it) {
            if (it->first == key) {
                table[index].erase(it);
                return;
            }
        }
    };
};
```